

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences

PROJECT SYNOPSIS

BCSP-064

DEVELOPMENT OF AN E-COMMERCE WEB APPLICATION FOR ONLINE SHOPPING

Particulars	Details
Name of Student	Kailash Kumar Jha
Enrolment Number	2400767095
Programme	Bachelor of Computer Applications (BCA)
Course Code	BCSP-064
Study Centre Code	07162P
Regional Centre Code	RCD1
E-mail	Provided in the submitted copy
Mobile Number	Provided in the submitted copy
Name of Project Guide	Shankar Jha
Designation of Guide	Backend Lead (Software Engineer)
Organisation	Wizcart Technologies LLC

Signature of the Student	Signature of the Guide
 Date:	 Date:

PROJECT SYNOPSIS

1. Title of the Project

DEVELOPMENT OF AN E-COMMERCE WEB APPLICATION FOR ONLINE SHOPPING

The project is referred to throughout by the working name **ShopSphere**.

2. Introduction and Objectives of the Project

Small and medium-sized retailers wishing to sell online must choose between a hosted platform that charges a subscription and a share of every transaction, and a self-hosted system that must be built and maintained. This project takes the second route and delivers a complete, tested e-commerce web application on the MERN stack — MongoDB, Express.js, React and Node.js — with a public storefront for customers and a back-office dashboard for the store administrator.

The objectives of the project are as follows.

1. To build a responsive customer interface that works on desktop, tablet and mobile without a separate mobile site.
2. To implement stateless authentication with JSON Web Tokens, storing passwords only as bcrypt hashes.
3. To provide catalogue search, category and price filtering, sorting and pagination, served by database indexes rather than collection scans.
4. To guarantee that concurrent checkouts cannot oversell stock, by making the stock check and the stock decrement a single atomic database operation.
5. To recompute every monetary amount on the server, so that a tampered client cannot alter what it pays.
6. To enforce role-based access control, so that a customer account cannot reach administrative functions or another customer's orders.
7. To provide an administrative dashboard with sales analytics, catalogue maintenance, order fulfilment and customer account administration.
8. To document every phase of the software development life cycle in accordance with the BCSP-064 guidelines.

3. Project Category

Category: RDBMS / Database Management System — implemented with a NoSQL (document-oriented) database, delivered as an Internet / Web Technologies application.

The project is data-centric: its core concern is the definition, indexing, querying and integrity of persistent data across four collections, together with the concurrency control needed to keep that data correct. It is implemented on MongoDB, which appears in the list of permitted database engines in Section VII of the BCSP-064 guidelines, and is delivered over web technologies (React, Express, REST), which appear in the same list. The project uses neither Visual Basic with MS-Access nor C or C++, both of which the guidelines prohibit for database projects.

4. Analysis

4.1 Context Diagram (DFD Level 0)

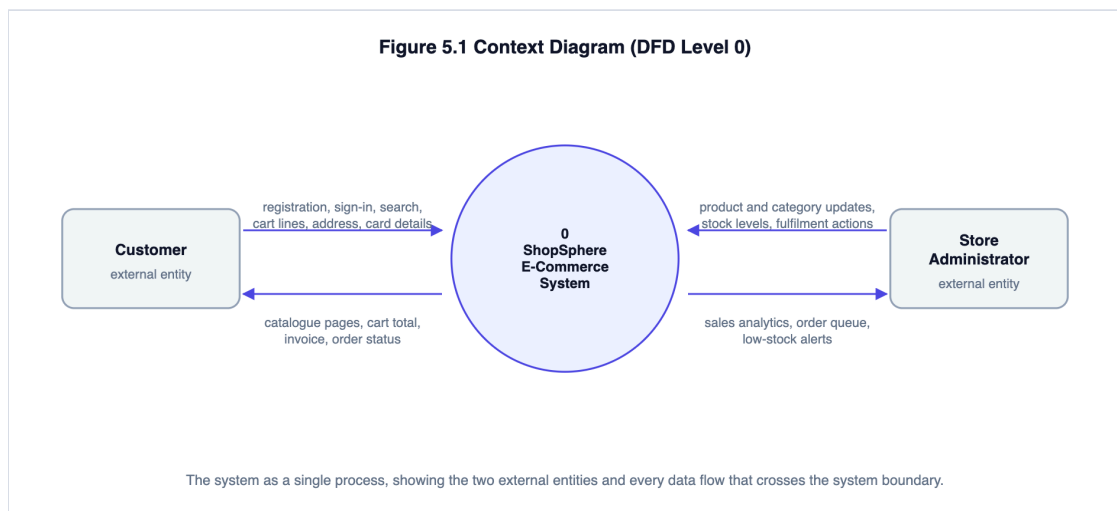


Figure S1: Context diagram — the system boundary and its two external entities

4.2 Data Flow Diagram — Level 1

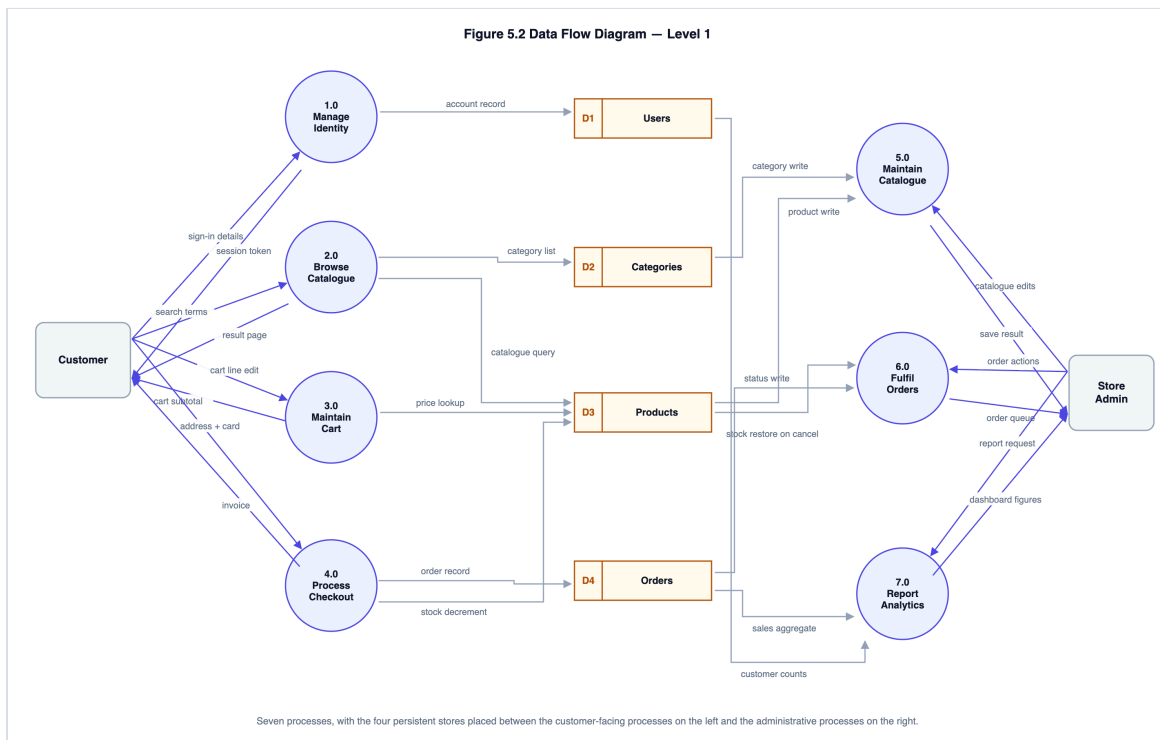


Figure S2: Level 1 DFD — seven processes and four data stores

4.3 Data Flow Diagrams — Level 2

Two processes are decomposed to the second level: the checkout engine, because it carries the system's most important correctness requirements, and catalogue browsing, because it is where untrusted input meets the database.

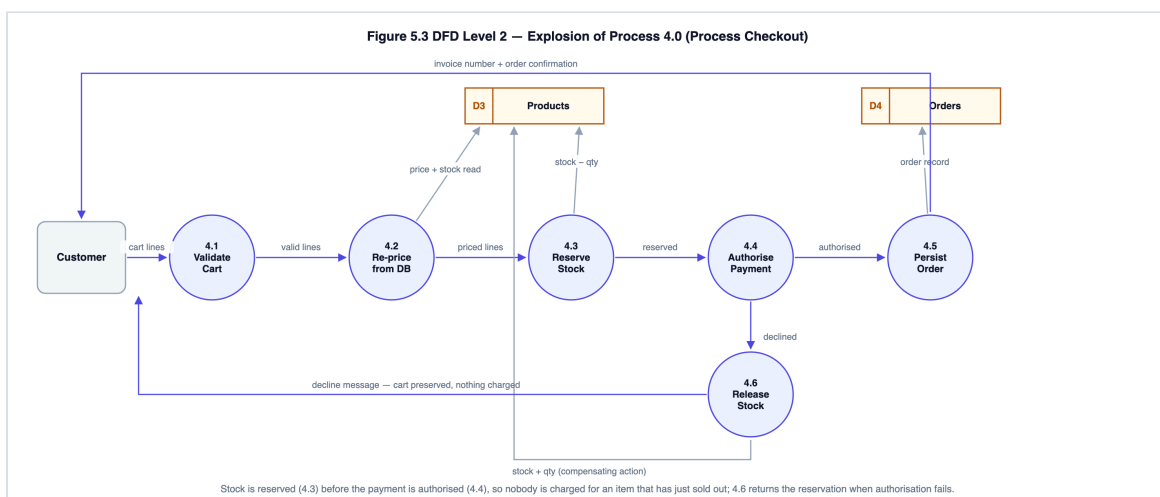


Figure S3: Level 2 DFD — explosion of process 4.0, Process Checkout

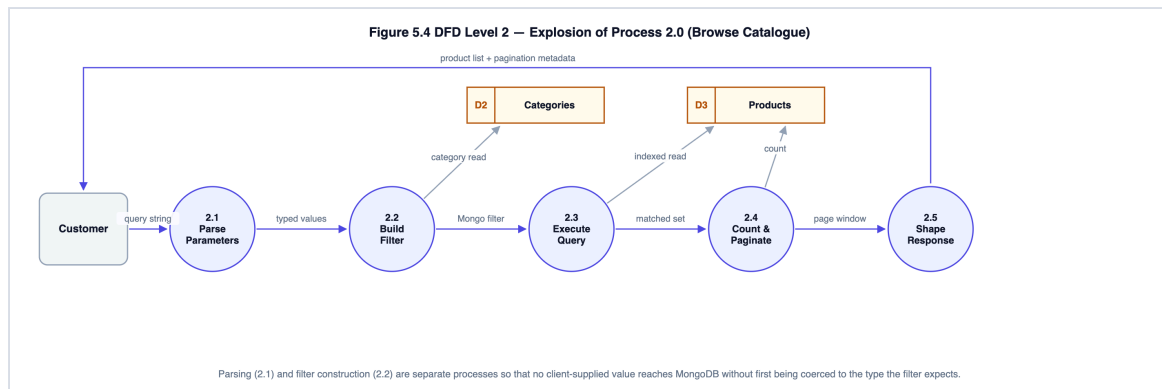


Figure S4: Level 2 DFD — explosion of process 2.0, Browse Catalogue

4.4 Entity–Relationship Diagram

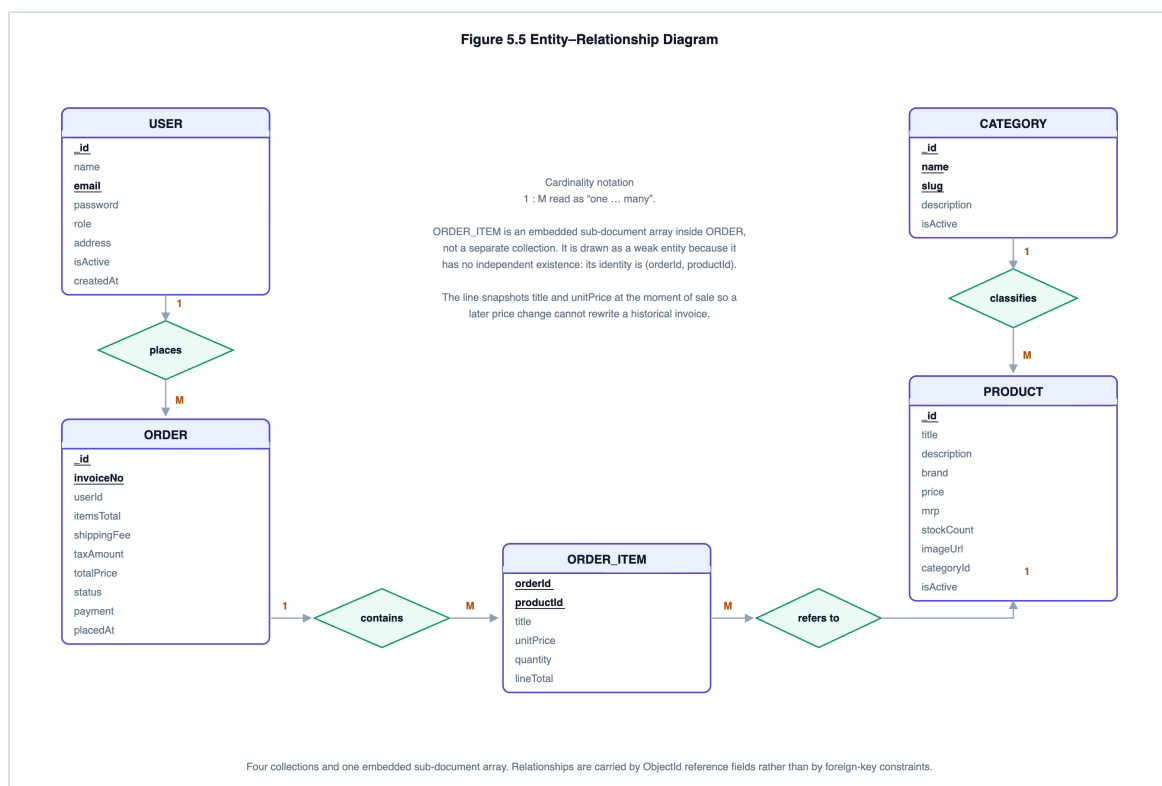


Figure S5: Entity–relationship diagram. ORDER_ITEM is a weak entity embedded within ORDER.

4.5 Use Case Diagram

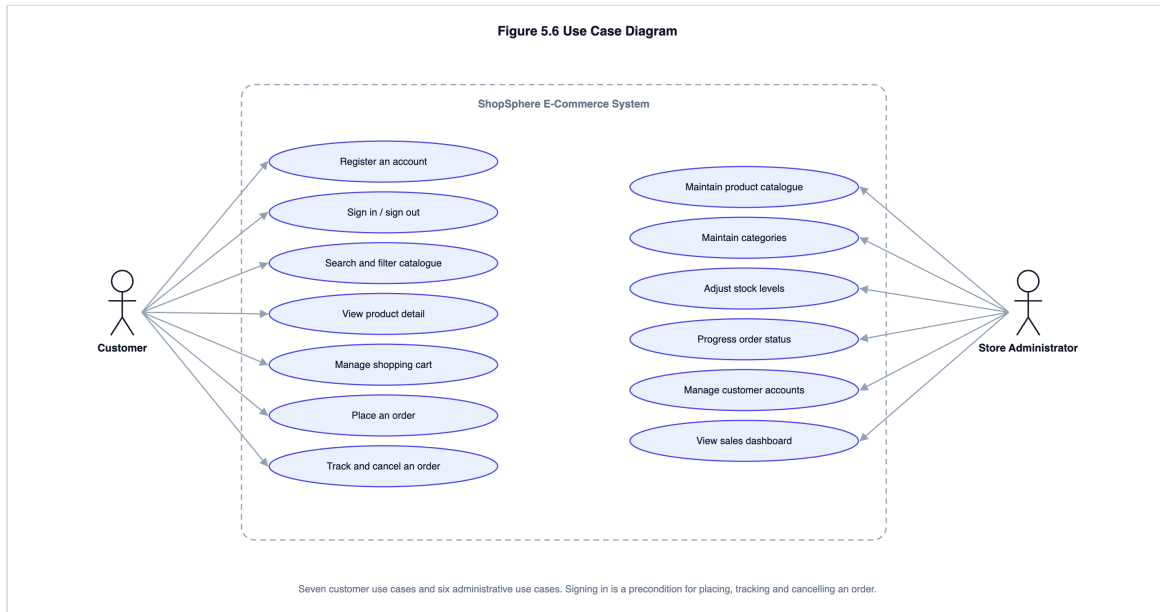


Figure S6: Use case diagram — seven customer and six administrative use cases

4.6 Database Design

The database comprises four collections. Relationships are carried by ObjectId reference fields; MongoDB enforces no foreign-key constraints, so referential integrity is maintained in application code.

Table S3: Collections, fields and indexes

Collection	Principal fields	Indexes
users	_id, name, email (unique), password (bcrypt hash), role {customer admin}, address, isActive, lastLoginAt	Unique on email
categories	_id, name (unique), slug (unique), description, isActive	Unique on name; unique on slug
products	_id, title, description, brand, price, mrp, category (ref), stockCount, imageUrl, rating, isActive	Weighted text index on (title 10, brand 5, description 1); compound (category, price); createdAt descending; isActive
orders	_id, invoiceNo (unique), user (ref), items[] {product, title, unitPrice, quantity, lineTotal}, itemsTotal, shippingFee, taxAmount, totalPrice, shippingAddress, payment, status, statusHistory[]	Unique on invoiceNo; compound (user, placedAt desc); compound (status, placedAt desc)

5. Complete Structure of the Project

5.1 Number of Modules and Their Description

The system comprises five functional modules beneath a single application shell.

Table S4: Modules and their scope

#	Module	Description	Endpoints
M1	Identity Management	Registration with a password policy, credential verification, JWT issue and verification, role-based route protection, profile and default-address maintenance, password change.	6
M2	Catalogue Management	Product and category storage; keyword search over a weighted text index; filtering by category, price band and availability; five sort orders; pagination; administrator create, amend and soft-delete.	7
M3	Shopping Cart	Browser-resident cart mirrored to local storage; add, change quantity and remove; per-item and per-order quantity caps; survives a page reload.	Client-side
M4	Checkout and Order Processing	Server-side re-pricing from stored prices; atomic stock reservation; simulated card gateway with Luhn and expiry validation, plus cash on delivery; invoice generation; order history; cancellation with stock restoration.	5
M5	Store Administration	Analytics dashboard with a fourteen-day revenue series, order status breakdown, best sellers and low-stock alerts; order fulfilment governed by a state machine; customer account activation.	9

5.2 Data Structures

Table S5: Principal data structures and the reason for each

Data structure	Where used	Why it is the right choice
Hash map (JavaScript Map)	Cart normalisation, merging duplicate lines for the same product; joining aggregation results to the day series in the dashboard.	Constant-time lookup by key, which turns what would be a nested scan into a single pass.
Array of sub-documents	<code>order.items</code> — the order lines embedded in each order document.	Order lines are written once and always read with their order, so embedding avoids a join on the most frequent order read.
B-tree index	Every <code>_id</code> , the unique indexes on email, slug and invoice number, and the compound (category, price) and (user, placedAt) indexes.	Logarithmic lookup; supports both equality and range predicates, and can satisfy a sort without a separate sort stage.
Inverted index (MongoDB text index)	Keyword search over product title, brand and description.	Maps each term to the documents containing it, giving relevance-ranked search without scanning the collection.
Finite state machine (transition table)	Order status, declared once as a frozen object on the Order model.	Makes every legal transition explicit in one place, so no code path can produce an invalid state.
Reducer over an immutable state array	Client-side cart state.	Every mutation is followed by the same persistence step; a single switch keeps the transitions in one readable place.
Frozen configuration object	<code>PRICING_RULES</code> , sort-key whitelist, environment configuration.	Business constants are defined once and cannot be mutated at run time.

5.3 Process Logic for Each Module

M1 — Identity Management. Registration validates the name, email and password against the stated policy, checks that the email is unused, and creates the account with the role forced to `customer` — the role is never read from the request body, which is what prevents privilege escalation at registration. A pre-save hook hashes the password with `bcrypt` at a work factor of twelve. Sign-in looks the account up by a normalised email address, compares the supplied password in constant time, and returns one generic message for both an unknown address and a wrong password so that registered addresses cannot be enumerated. Every protected request extracts the bearer token, verifies its signature and expiry, and then re-reads the account from the database rather than trusting the token payload, so that deactivating an account takes effect on its very next request.

M2 — Catalogue Management. The listing endpoint passes the query string to a filter builder that coerces every value to the type the filter expects: an identifier only if it parses as an `ObjectId`, a price bound only if it parses as an integer, a sort key only if it appears in a whitelist, and a page size capped at forty-eight. A raw string is never handed to MongoDB, which is what prevents operator injection through a query parameter. A keyword search uses the weighted text index and, unless an explicit order is requested, is returned ranked by relevance. An inverted price band — minimum above maximum — is discarded rather than applied, because applying it would return an empty page with no explanation. Deletion is a soft delete that clears `isActive`, so historical orders retain a resolvable product reference.

M3 — Shopping Cart. The cart is held in browser memory and mirrored to local storage so it survives a page reload. Quantities are capped per product and the number of distinct products is capped per cart. Both caps are re-applied on the server, because the client is not a security boundary. The totals displayed are explicitly labelled an estimate: the binding figure is the one the server returns at checkout.

M4 — Checkout and Order Processing. The submitted cart is treated as nothing more than a list of product identifiers and quantities. Duplicate lines for the same product are

merged before the quantity cap is applied, so splitting a line cannot bypass it. Every price is then re-read from the database and every total recomputed. Stock is reserved by a conditional atomic update — `findOneAndUpdate` with a filter requiring sufficient stock and an `$inc` that decrements it — which makes the check and the decrement a single indivisible operation and therefore closes the race window in which two concurrent checkouts could both succeed. Reservation happens **before** payment authorisation, so a customer is never charged for an item that has just sold out; if authorisation then fails, every reservation already taken is released as a compensating action. Where the deployment is a replica set the whole sequence runs inside a real multi-document transaction; on a standalone server it falls back to compensation. Finally the order is written with a cryptographically random invoice number and a snapshot of the title and unit price of every line, so that a later catalogue price change cannot rewrite a historical invoice.

M5 — Store Administration. Dashboard figures are computed by MongoDB aggregation pipelines rather than by loading orders into the application, so the work scales with the database rather than with application memory. Calendar-day buckets are resolved in the store’s configured timezone, not in UTC. Order status changes are validated against the transition table declared on the Order model; a cancellation returns stock to the catalogue and marks a paid order as refunded, and a cash-on-delivery order is marked paid when it reaches DELIVERED. An administrator cannot deactivate their own account, because the screen needed to undo it would become unreachable.

5.4 Testing Details

Testing is planned at three levels, with test cases designed and reported separately for each as the guidelines require.

Table S6: Planned testing at three levels

Level	Scope	Approach	Planned cases
Unit testing	Individual functions and schemas in isolation — Luhn validation, expiry checking, query-string parsing, cart normalisation, pricing arithmetic, calendar helpers, schema validation and hooks.	Jest. Database-touching cases run against an in-memory MongoDB instance so that real index and constraint behaviour is exercised rather than mocked.	≈ 55

Level	Scope	Approach	Planned cases
Integration testing	Complete request paths — router, validation chain, middleware, controller, service, model and database — for all twenty-seven endpoints.	Jest with Supertest driving the Express application over real HTTP in-process. Nothing is stubbed.	≈ 65
System testing	Whole user journeys in the order a real visitor performs them: registration → search → add to cart → checkout → fulfilment; catalogue narrowing; cancel and rebuy; administrator catalogue lifecycle; session lifecycle across a password change.	Jest. Each step uses only data returned by the previous step, so a break anywhere fails the case.	5

Specific properties that will be asserted rather than assumed include: that concurrent checkouts cannot oversell the last unit; that a client-supplied price is ignored; that a declined payment leaves stock unchanged and writes no order; that no card number or CVV is persisted; that a customer cannot read another customer's order; and that deactivating an account takes effect immediately.

5.5 Reports Generation

The system generates the following reports and outputs. Each is produced from live data at the moment it is requested; none is pre-computed or cached.

Table S7: Reports generated by the system

Report / output	Audience	Tentative contents
Sales summary	Administrator	Total revenue, order count, units sold and average order value across all realised orders (confirmed, shipped and delivered).
Fourteen-day revenue trend	Administrator	Revenue and order count per calendar day for the last fourteen days, plotted as an area chart, with days of no trading shown as zero rather than omitted.
Order status breakdown	Administrator	Count of orders in each of the five states, plotted as a horizontal bar chart.
Best-selling products	Administrator	Top five products by units sold, with the revenue each generated.
Low-stock alert	Administrator	Every active product at or below five units, ordered by stock ascending, with an out-of-stock flag.
Order queue	Administrator	All orders with invoice number, customer, date, item count, total, payment method and status; filterable by status and paginated.
Customer register	Administrator	All accounts with name, email, role, registration date and activation status; searchable by name or email.
Customer order history	Customer	Own orders, newest first, with invoice number, date, total, status and item summary.

Report / output	Audience	Tentative contents
Order detail / invoice	Customer and administrator	Invoice number, order date, itemised lines with unit price and line total, subtotal, delivery charge, tax, payable total, delivery address, payment method with the last four digits of the card, and the full status timeline.

6. Tools, Platform and Hardware / Software Requirements

6.1 Tools and Platform

Table S8: Tools and platform

Layer	Technology
Front end	React 18, react-router-dom 6, React Context API, Axios, Recharts, Vite 5, CSS3 (Grid, custom properties)
Back end	Node.js 22, Express 4, Mongoose 8, jsonwebtoken, bcryptjs, express-validator, helmet, express-mongo-sanitize, express-rate-limit, cors, morgan
Database	MongoDB 7 Community Server, with text and compound indexes and the aggregation framework
Testing	Jest 29, Supertest 7, mongodb-memory-server 10
Development tools	Visual Studio Code, Git, Postman, MongoDB Compass, Chrome DevTools
Architecture	Decoupled three-tier: React SPA → stateless REST API → document database

6.2 Hardware Requirements

Table S9: Hardware requirements

Component	Development machine	Deployment server	Client device
Processor	Dual-core 2.0 GHz or better	One virtual CPU	Any
Memory	8 GB (4 GB minimum)	1–2 GB	2 GB
Disk	10 GB free	10 GB, growing with order history	Negligible
Network	Broadband	Public IP address or reverse proxy	Any internet connection
Display	1366 × 768 or better	Not applicable	360 px width or wider

6.3 Software Requirements

Table S10: Software requirements

Component	Development	Deployment	Client
Operating system	Windows 10+, macOS 12+ or Ubuntu 20.04+	Any Linux with a supported Node.js build	Any
Runtime	Node.js 18 LTS or later, npm 9+	Node.js 18 LTS or later	Not applicable

Component	Development	Deployment	Client
Database	MongoDB 6+ Community Server	MongoDB 6+; a replica set is recommended so that real multi-document transactions are available	Not applicable
Browser	Chrome or Firefox with developer tools	Not applicable	Chrome 90+, Firefox 88+, Safari 14+ or Edge 90+
Other	Git, Visual Studio Code, Postman	A process supervisor (pm2 or systemd); TLS termination at the proxy	JavaScript and local storage enabled

7. Is This Project Being Done for Any Industry or Client?

No.

This project is not being carried out for any industry, client or organisation. It is an independent academic project, formulated by the student in consultation with the project guide as permitted by Section III of the BCSP-064 guidelines, which state that it is not mandatory for a student to work on a real-life project and that the student may formulate a project problem with the help of the guide. The problem domain is modelled on the needs of a small retailer, but no particular retailer is involved, no client data is used, and no confidentiality restriction applies to the source code, which is reproduced in full in the project report.

8. Future Scope and Further Enhancement

The architecture is deliberately decoupled so that the enhancements below can be made without rebuilding the system.

- **Product variants.** A `variants` array on the product document, each variant carrying its own SKU, attributes and stock count, with the atomic reservation operating on a positional array element. Required before apparel or footwear can be sold properly.
- **Live payment gateway.** Replacing the simulated gateway with a certified integration, in which the browser posts card details directly to the gateway and the application only ever sees a token. The module boundary is drawn so this touches one file.
- **Server-side cart.** Persisting the cart against the account so it follows the customer between devices.
- **Email and SMS notification.** Transactional despatch on order placement and each status change, queued so that a provider outage cannot fail a checkout.

- **Customer reviews.** A reviews collection with a verified-purchase check and incremental update of the product rating.
- **Discount codes and promotions.** Validity rules applied during the server-side re-pricing step, so a discount cannot be forged by the client.
- **Image upload pipeline.** Multipart upload, server-side resizing and delivery through a content delivery network.
- **Native mobile application.** Because the server is a headless REST API, a React Native client can consume the same endpoints with no server change.
- **Recommendation engine.** Collaborative filtering over the co-purchase information already present in the order history, replacing the current same-category query.
- **Dedicated search engine.** Elasticsearch or MeiliSearch for typo tolerance, synonyms and faceted counts, none of which a MongoDB text index provides.
- **Multi-vendor marketplace.** Seller accounts, per-seller catalogues, commission calculation and split settlement.
- **Progressive web application.** A service worker for offline catalogue browsing and installability on mobile.